

1. Setup and Automation (Task 1.1)

The pom.xml build script was modified to integrate PIT mutation testing using the pitest-maven plugin. Mutation analysis is scoped to org.apache.commons.collections4.*, while the coursework suites (M*Test and D*Test) are excluded from the default developer run. A \${pit.report.dir} property allows the report directory to be overridden from the command line so that the same build configuration can be reused across all runs. Mutation testing is executed using: mvn clean compile test-compile org.pitest:pitest-maven:mutationCoverage. To obtain the baseline required for Task 1.1, PIT was executed on the original Apache Commons Collections project using the unmodified developer JUnit test suite. This baseline run covered 267 classes and reported 48% line coverage (6321/13194), 41% mutation coverage (3499/8449), and 85% test strength. The archived developer PIT results included in the final submission are reported separately in Appendix A.

2. Representative Killed Mutant (Task 1.2)

A representative killed mutant was selected from the archived baseline developer PIT report in AbstractHashMap, constructor <init>(IF)V, line 142. PIT reports this as a NegateConditionalsMutator, which changes the guard if (initialCapacity < 0) to if (initialCapacity >= 0). As a result, the constructor would throw an IllegalArgumentException for valid capacities instead of only for negative ones, preventing the map from being constructed. This mutant is killed by TransformedMapTest.testFactory_decorateTransform. The test constructs a TransformedMap wrapping a standard HashMap, which exercises the affected constructor with a valid capacity. In the original program, this construction succeeds, but under the mutation the constructor throws immediately, causing the test to fail and therefore killing the mutant. Full mutant details are reported in Appendix B.

3. Improving Mutation Score (Task 1.3)

The mutant selected for this task occurs in LazyMap.get at line 176, where PIT reports a NullReturnValsMutator. This mutation replaces the return value of the method with null. In the original developer test suite, this mutant was already killed by MapUtilsTest.testLazyMapTransformer, however, the directly relevant test, LazyMapTest.testMapGet(), contained no assertions and only a TODO comment from an earlier refactoring. This meant that testMapGet() provided no independent oracle for the behaviour of LazyMap.get, and would not have detected the mutation on its own. To address this, testMapGet() was rewritten so that both execution paths of LazyMap.get are exercised explicitly. The strengthened test first checks that a missing key triggers the factory and that the returned value matches the value created by the factory. It then verifies that an existing key returns the value already stored in the backing map rather than null. These assertions make the returned value part of the test oracle, meaning testMapGet() now also kills this mutant independently. In the final archived developer PIT run, the mutant is recorded as KILLED by LazyMapTest.testMapGet, contributing to a final project mutation coverage of 38% (3232/8451). The PIT report entry and the modified test method are provided in Appendix C.

4. Improving Coverage and Mutation Score (Task 1.4)

For Task 1.4, the selected mutation occurs in PeekingIterator.hasNext at line 98, where PIT reports a BooleanTrueReturnValsMutator. This mutation replaces the boolean return value with true, affecting the branch executed when the iterator has already been exhausted. In the original developer test suite, the method testIteratorExhausted() checked that hasNext() returned false before calling peek(), and then verified that peek() returned null. However, the behaviour of hasNext() after peek() was not explicitly verified once the iterator had entered the exhausted state. As a result, the exhausted-state branch of hasNext() was not fully exercised by the original tests. To address this limitation, PeekingIteratorTest.testIteratorExhausted() was strengthened by adding an additional assertion immediately after the peek() call: assertFalse(it.hasNext()); This assertion ensures that after the iterator becomes exhausted, hasNext() continues to return false. If the mutation forces the method to always return true, the assertion fails, thereby killing the mutant. The strengthened test also verifies that calling element() still throws NoSuchElementException, confirming that the iterator remains in the exhausted state. This improves behavioural coverage by explicitly exercising the exhausted-state execution path. In the final archived developer PIT run, this mutant is recorded as KILLED by PeekingIteratorTest.testIteratorExhausted(). The corresponding PIT report entry and the modified test method are provided in Appendix D.

5. LLM-Generated Mutation Suites (Task 1.5)

Three LLM-generated test suites were constructed: M16, M32, and M64, containing 16, 32, and 64 tests, respectively. Each suite was executed independently by setting `targetTests` to the corresponding suite class and overriding the report directory with `-Dpit.report.dir`, ensuring that each PIT execution produced a separate mutation report. The tests were generated with the assistance of ChatGPT, which was used to propose candidate JUnit 4 tests for Apache Commons Collections classes. The generated tests were not accepted directly; instead, they were manually compiled, refined, and validated before inclusion in the final suites.

The generation process followed an iterative workflow:

1. Request a small number of focused JUnit tests targeting a specific class or behavioural scenario;
2. Compile the generated tests and resolve syntax, import, generics, or assertion issues;
3. Execute Maven and PIT to verify that the tests were discoverable and executed correctly;
4. Remove weak, redundant, flaky, or highly overlapping tests;
5. Repeat the process until final suites of 16, 32, and 64 LLM-generated tests were obtained.

Representative prompts guided the model toward behaviourally meaningful tests rather than trivial getters or setters.

For example:

“Generate JUnit 4 tests for Apache Commons Collections 4.4 that target edge cases and observable behaviour rather than trivial getters or setters.”

To reduce overlap with the existing developer test suite, prompts emphasised edge cases such as null handling, iterator exhaustion, decorator behaviour, cached values, and exception paths. Generated tests were manually reviewed and discarded if they appeared structurally too similar to existing developer tests.

Suite	n	Mutated classes	Line coverage	Mutation coverage	Test strength	Timed out
M16	16	267	4% (508/13194)	3% (214/8451)	71% (214/300)	27
M32	32	267	3% (367/13194)	2% (207/8451)	74% (207/278)	27
M64	64	267	3% (422/13194)	3% (243/8451)	91% (243/266)	27

Table E1 - PIT mutation metrics for the LLM-generated test suites.

Observation: Among the LLM suites, M64 achieves the highest test strength (91%) and the highest number of killed mutants (243/8451). However, all three LLM suites remain relatively lightweight and therefore cover only a small fraction of the overall code base. M32 also shows slightly lower line coverage than M16 (3% vs 4%), indicating that suite composition and the specific classes targeted by the generated tests have a greater effect on coverage than suite size alone.

6. Developer Sample Suites (D16/D32/D64) (Task 1.6)

To compare the LLM-generated suites with subsets of the original developer tests, three developer sample suites were constructed: D16, D32, and D64. Each suite is implemented as a JUnit suite class that includes 16, 32, and 64 existing developer test classes respectively. Unlike the LLM suites, which contain individual generated test methods, these developer suites are composed of entire existing test classes. As a result, their effectiveness depends strongly on which classes are included in the sample rather than strictly on the total number of classes. Consequently, performance is not necessarily monotonic with suite size. Each suite was executed independently by setting `targetTests` to the corresponding suite class and running PIT with the same configuration used for the other experiments.

Suite	n	Mutated classes	Line coverage	Mutation coverage	Test strength	Timed out
D16	16	267	23% (2972/13194)	21% (1739/8451)	87% (1739/1990)	27
D32	32	267	12% (1537/13194)	8% (717/8451)	77% (717/927)	27
D64	64	267	15% (2030/13194)	13% (1136/8451)	89% (1136/1275)	28

Table F1 - PIT mutation metrics for sampled developer test suites.

Observation: Among the sampled developer suites, D16 achieves the highest mutation coverage (21%) and the highest line coverage (23%), despite containing fewer test classes than the larger samples. This occurs because the selected

developer classes in the D16 suite exercise behaviour that is particularly mutation relevant. Since the suites include entire test classes rather than individual test methods, increasing the number of classes does not guarantee higher coverage or mutation scores. Instead, the behavioural richness of the included tests plays a more important role in determining mutation effectiveness.

7. Test Suite Diversity with I-TSDm (Task 1.7)

To quantify structural diversity between the sampled test suites, the coursework uses the I-TSDm diversity metric, which is based on the Normalized Compression Distance (NCD). The metric measures how different the test cases are from each other based on the compressibility of their source text. For the developer suites (D-suites), the source text of entire test classes was used, since these suites are composed of existing JUnit test classes. For the LLM-generated suites (M-suites), diversity was measured using the bodies of individual `@Test` methods, because the generated suites consist of individual test cases rather than whole classes. Compression was performed using gzip over UTF-8 encoded source text, and the multiset NCD was approximated using the heuristic removal chain method described in the literature for scalable diversity estimation.

Suite	n	Approx. I-TSDm
D16	16	0.981
D32	32	0.951
D64	64	0.986
M16	16	0.910
M32	32	0.905
M64	64	0.946

Table G1 - Approximate I-TSDm diversity scores for developer and LLM-generated test suites.

Interpretation: Values closer to 1 indicate higher structural diversity within the test suite. The sampled developer suites achieve higher diversity scores than the LLM-generated suites, reflecting the fact that developer tests span a wider range of subsystems and behaviours within Apache Commons Collections. In contrast, the LLM-generated suites remain moderately diverse but are more behaviourally focused, often targeting similar categories of edge cases such as iterator exhaustion, null handling, and collection utilities. As a result, although the generated suites contribute useful mutation-relevant tests, their internal diversity is lower than that of the sampled developer suites.

8. Compare and Contrast (Task 1.8)

The archived results allow a comparison between three categories of test suites: the full developer suite, the sampled developer suites (D16/D32/D64), and the LLM-generated suites (M16/M32/M64). The archived developer suite used in the final submission remains the strongest overall configuration, achieving 44% line coverage, 38% mutation coverage, and 85% test strength. This reflects the cumulative effect of a large number of carefully engineered tests developed over time. When comparing reduced suites of similar size, the sampled developer suites consistently outperform the LLM-generated suites on raw coverage and mutation score. For example, D16 achieves 23% line coverage and 21% mutation coverage, whereas the corresponding LLM suite M16 achieves only 4% line coverage and 3% mutation coverage. This indicates that developer tests exercise a broader range of behaviours across the code base. However, the LLM suites still demonstrate useful properties. Although they cover only a small fraction of the project, they are compact and inexpensive to execute, and the strongest generated suite (M64) achieves 91% test strength, indicating that the mutants it does cover are often checked with relatively strong assertions. Another observation from the archived results is that suite size alone does not determine mutation effectiveness. For instance, D16 outperforms both D32 and D64, while M64 performs better than M16 and M32 among the generated suites. These results suggest that suite composition, behavioural diversity, and assertion quality are at least as important as the number of tests. A hybrid strategy is most effective: the developer suite provides the regression backbone, strengthened tests address mutation weaknesses, and LLM suites contribute complementary edge-case tests. The PIT reports used in this analysis can be reproduced using the commands listed in Appendix H, and a complete summary of the mutation testing results for all runs is provided in Appendix I.

APPENDICES

Appendix A - PIT Developer Runs

Run	Mutated classes	Line coverage	Mutation coverage	Test strength
Developer baseline	267	48% (6321/13194)	41% (3499/8449)	85% (3499/4102)

Table A1 - Original developer baseline (pre-modification).

Run	Mutated classes	Line coverage	Mutation coverage	Test strength
Archived developer run	267	44% (5861/13194)	38% (3232/8451)	85% (3232/3804)

Table A2 - Archived developer run (post-modification).

Appendix B - PIT Evidence for the Selected Killed Mutant (Task 1.2)

Class	org.apache.commons.collections4.map.AbstractHashMap
Method	<init>(IF)V
Line	142
Mutator	NegateConditionalsMutator
Mutation description	negated conditional
Status in archived dev run	KILLED
Killing test	TransformedMapTest.testFactory_decorateTransform

Table B1 - PIT report entry for the representative killed mutant selected in Task 1.2.

Appendix C - PIT Evidence and Modified Test for Task 1.3

Class	org.apache.commons.collections4.map.LazyMap
Method	get
Line	176
Mutator	NullReturnValsMutator
Mutation description	replaced return value with null for LazyMap::get
Status in archived dev run	KILLED
Killing test	LazyMapTest.testMapGet
Why the stronger test matters	It explicitly distinguishes the 'existing key returns stored value' behaviour from a spurious null return.

Table C1 - PIT report entry for the selected mutant discussed in Task 1.3.

Listing C1 - Modified LazyMapTest.testMapGet() used to kill the selected mutant.

```
@Override
public void testMapGet() {
    // Exercise LazyMap.get() in both branches:
    // (a) missing key -> factory creates value and the method returns that value
    // (b) existing key -> returns the value already stored in the backing map
    final Map<Object, Number> map = lazyMap(new HashMap<Object, Number>(), oneFactory);
    assertEquals(0, map.size());

    final Number created = map.get("Five");
    assertEquals(1, created);
    assertEquals(1, map.size());

    // Same logical key via a different String instance must return the same cached value
    final Number cached = map.get(new String(new char[] { 'F', 'i', 'v', 'e' }));
    assertEquals(1, cached);
    assertEquals(created, cached);
    assertEquals(1, map.size());

    // Existing-key branch
    map.put("Six", 6);
    assertEquals(6, map.get("Six"));
}
```

Appendix D - PIT Evidence and Modified Test for Task 1.4

Class	org.apache.commons.collections4.iterators.PeekableIterator
Method	hasNext
Line	98
Mutator	BooleanTrueReturnValsMutator
Mutation description	replaced boolean return with true for PeekableIterator::hasNext
Status in archived dev run	KILLED
Killing test	PeekableIteratorTest.testIteratorExhausted
Why the stronger test matters	The test now checks that after exhaustion and peek(), hasNext() stays false and element() still throws.

Table D1 - PIT report entry for the mutant discussed in Task 1.4.

Listing D1 - Modified PeekableIteratorTest.testIteratorExhausted() used to kill the selected mutant.

@Test

```
public void testIteratorExhausted() {
    final PeekableIterator<E> it = makeObject();
    it.next();
    it.next();
    it.next();
    assertFalse(it.hasNext());
    assertNull(it.peek());
    // After peek() on an exhausted iterator, the internal "exhausted" flag is set; hasNext() should stay false
    assertFalse(it.hasNext());

    try {
        it.element();
        fail();
    } catch (final NoSuchElementException e) {
        // expected
    }
}
```

Appendix H - PIT Commands and Report Locations.

Open PIT reports (Linux):

```
xdg-open pitReports/dev/index.html

xdg-open pitReports/sample-16/index.html

xdg-open pitReports/sample-32/index.html

xdg-open pitReports/sample-64/index.html

xdg-open pitReports/llm-16/index.html

xdg-open pitReports/llm-32/index.html

xdg-open pitReports/llm-64/index.html
```

Command templates used with the corrected pom.xml:

Developer baseline:

```
mvn clean compile test-compile org.pitest:pitest-maven:mutationCoverage
```

LLM suite example (M16):

```
mvn -Dpit.report.dir=pitReports/llm-16 -DtimestampedReports=false -
DexcludedTestClasses= -DtargetTests=org.apache.commons.collections4.M16Test clean
compile test-compile org.pitest:pitest-maven:mutationCoverage
```

Developer sample example (D16):

```
mvn -Dpit.report.dir=pitReports/sample-16 -DtimestampedReports=false -
DexcludedTestClasses= -DtargetTests=org.apache.commons.collections4.D16Test clean
compile test-compile org.pitest:pitest-maven:mutationCoverage
```

Appendix I - Summary of All PIT Runs.

Report	Classes	Line coverage	Mutation coverage	Test strength	Total muts	KILLED	SURVIVED	NO_COVERAGE	TIMED_OUT
Developer suite (Dev)	267	44% (5861/13194)	38% (3232/8451)	85% (3232/3804)	8451	3205	572	4647	27
LLM suite (M16)	267	4% (508/13194)	3% (214/8451)	71% (214/300)	8451	187	86	8151	27
LLM suite (M32)	267	3% (367/13194)	2% (207/8451)	74% (207/278)	8451	180	71	8173	27
LLM suite (M64)	267	3% (422/13194)	3% (243/8451)	91% (243/266)	8451	216	23	8185	27
Developer sampled (D16)	267	23% (2972/13194)	21% (1739/8451)	87% (1739/1990)	8451	1712	251	6461	27
Developer sampled (D32)	267	12% (1537/13194)	8% (717/8451)	77% (717/927)	8451	690	210	7524	27
Developer sampled (D64)	267	15% (2030/13194)	13% (1136/8451)	89% (1136/1275)	8451	1108	139	7176	28